

Smart College AI

Project Review Document

VERSION	2.5.0
BACKEND	Python 3 (built-in HTTP server)
DATABASE	Supabase / PostgreSQL — optional cloud sync
DEFAULT PORT	8080
PREPARED	18 September 2026

A full-stack Academic Management & Digital Twin Web Platform. It combines a Python HTTP backend with an HTML + Tailwind + Vanilla JS frontend, 20-student and 20-staff datasets, a rule-based AI chatbot, and optional Supabase cloud sync.

Prepared for project review · Location: `D:\test\smart-college-management` · Document generated for academic presentation.

What this platform delivers

Smart College AI is a browser-based digital-twin platform that mirrors a student's full academic life — attendance, grades, fees, certificates, faculty contact, and predictive health — through a single web workspace. It supports both students (11 dashboard tabs) and staff (8 admin tabs) with a shared AI chatbot that answers questions against live record data rather than a generic model. The backend runs on Python 3's built-in HTTP server; no external web framework is required. The database layer is optional: all features work locally from JSON/CSV sources, and a Supabase cloud connection activates when a user enters their API key.

The review should confirm: end-to-end student and staff journeys, authentication behavior, AI response accuracy, API contract completeness, data model alignment (supabase_schema.sql), test coverage (7 automated tests), and the security posture of using demo credentials in a development setting.

20 Students modeled across 6 departments	20 Staff members (HODs, faculty, deans)	9 Supabase tables with RLS + seed	7 Automated tests (all passing)
--	---	---	---

CONTENTS

Table of Contents

1	Project Overview & Architecture
2	Folder Structure & File Roles
3	End-to-End User Flow
4	Backend — server.py
5	Frontend — index.html & app.js
6	Data — Students, Staff, JSON / CSV
7	Database Schema — supabase_schema.sql
8	Dual Data Architecture
9	Authentication System
10	AI Chatbot Engine
11	API Endpoints
12	Key Features & Demonstrations
13	How To Run & Testing
14	Appendix — Source Maps

1 Project Overview & Architecture

Smart College AI serves as a digital mirror of a student's academic life. Its design goal is to let students check attendance, fees, grades, and faculty info from a browser; let staff mark attendance, approve certificates, and email cohorts; and let both groups ask a chatbot that pulls answers from their actual records. The platform is fully local-first, with an optional Supabase cloud layer that activates via a key input in Settings.

Architecture at a glance

Backend	Data (local)	Data (cloud)
HTML + Tailwind CSS + Vanilla JS + Material Symbols	All student/staff screens (single-page app via tab switching)	
Python 3 (http.server)	Auth, AI response, fee/attendance updates, email simulation	
JSON / CSV (data/)	20-student + 20-staff records; generated from CSV via generate_json.py	
Supabase PostgreSQL	9 tables with RLS policies; seed data included; optional live sync	

Version & deployment info

App Version **2.5.0** | Supabase project <https://aqfqdjhddfvypwccyp.supabase.co> | Default port **8080** (configured in `run.sh`). Backend runs as a single-file Python server with zero external dependencies; frontend pulls Tailwind and Material Symbols from CDN.

REVIEW NOTE: All references to Supabase URL, demo passwords (student@123 / staff@123), and student/staff IDs in this document are development/configuration values. Confirm they are replaced or isolated before any production or assessment deployment.

2 Folder Structure & File Roles

Every file has a single documented purpose. The tree is intentionally flat — only `static/` and `data/` branch deeper.

Path	Role	Key content / lines
PROJECT_DOCUMENT.md	Review doc (this source)	759 lines — full explanation of all sections
README.md	Quick guide	128 lines — capabilities, datasets, run instructions
server.py	Backend	867 lines — auth, AI chatbot, endpoints, JSON load
test_app.py	Tests	131 lines — 7 automated checks
run.sh	Startup	13 lines — sets PORT=8080, starts server.py
data/students_dataset.csv	Student CSV source	20 records (Aarav ... Aditi)
data/students.json	Student data (runtime)	36 918 bytes — full record objects
data/staff_dataset.csv	Staff CSV source	20 records (Dr. Rajesh Raman ...)
data/staff.json	Staff data (runtime)	8 374 bytes — profiles + courses
data/generate_json.py	Converter	CSV → JSON pipeline
supabase/supabase_schema.sql	DB schema	23 186 bytes — 9 table definitions + seed INSERTs
static/index.html	Frontend UI	83 349 bytes — all tabs (single page)
static/js/app.js	Frontend logic	57 677 bytes — init, login, tab switch, AI, API calls

Analogy from the source documentation: **server.py** = principal's office; **index.html** = building front door; **app.js** = internal wiring; **data/*.json** = filing cabinet.

3 End-to-End User Flow

This walkthrough maps the exact steps a student named **Aarav Sharma** (CSE, ID 23331a42001) takes from browser open to AI answer.

Step-by-step (from source)

Open browser	Go to <code>http://localhost:8080</code>
Login screen	Select Student / Staff tab; enter ID + password; or click 1-click demo pill (Aarav / Dr. Rajesh Raman)
Server verifies	POST <code>/api/auth/login</code> → server loads <code>students.json</code> → match <code>23331a42001</code> + <code>student@123</code> → return token + user object
Dashboard renders	<code>app.js</code> hides login, shows workspace, fetches <code>/api/students/23331a42001</code> , renders cards (91% attendance, 8.74 CGPA, █12,500 fees)
AI question	Type “ <i>Can I skip DBMS today?</i> ” → POST <code>/api/chat</code> → server finds DBMS ($28/36 = 77.8\%$) → calculates 75.7% if skipped → formatted response
Data can change	Staff marks attendance (<code>/api/attendance/mark</code>) → student pays fees (<code>/api/fees/pay</code>) → applies for cert (<code>/api/requests</code>) → writes to JSON or Supabase

4 Backend — server.py

The backend is a single Python file (867 lines) using `http.server.HTTPServer`. It does not import Flask, FastAPI, or Django — all routing is manual via `do_GET` / `do_POST`. The server reads all student/staff data from JSON at startup and writes changes back to disk (or to Supabase when configured).

Core functions (documented)

authenticate_user(role, identifier, password) — checks JSON files, supports ID or email, case-insensitive. Returns user object + token. **generate_ai_response(query, user_context, role)** — keyword-matching engine: finds subject attendance, calculates projection on skip, returns formatted Markdown with emoji. **send_email_to_students()** — SMTP if configured; else writes to `data/email_log.json`.

Key API routes

[illegible]

5 Frontend — index.html & app.js

Frontend structure (index.html)

One large HTML file (~1200 lines) containing every screen. Navigation is a sidebar; only one tab <div> is visible at a time (controlled by app.js via switchTab()). Styling uses Tailwind CSS via CDN; icons use Material Symbols via CDN. No build step required.

Student tabs (11)

Dashboard · AI Copilot · Attendance Ledger · Fees & Payments · CGPA & Marks · Certificates · Digital Twin · Student Dataset · Staff Directory · Settings. Each tab pulls data from /api/students or /api/staff via fetch.

Staff tabs (8)

Faculty Overview · My Courses · Attendance Marker · Certificate Approvals · Faculty AI Assistant · Student Dataset · Staff Dataset · Settings. Staff-specific AI chips replace student prompts after login.

Frontend logic (app.js)

~1350 lines. Core flow: initApp() (load data, check session) → performLogin() → switchTab() → render functions per tab → sendFullAiQuery() (typing animation → response bubble). State is kept in a JS object: authRole, authUser, students[], staff[], allRequests[], activeTab. Session stored in sessionStorage; Supabase key in localStorage. Fallback login validates locally if server is down (demonstration mode).

6 Data — Students, Staff, JSON / CSV

Student dataset (data/students.json)

20 records. Each record includes: student_id, password, full_name, department, email, cgpa, phone, semester, year, attendance_pct, fee_balance, fee_status, digital_twin_score, subjects[], active_requests[], recent_activities[].

Representative student records

23331a42001	23331a42002	23331a42013	23331a42003	23331a42005	23331a42006
23331a42001	Aarav Sharma	CSE	8.74	91%	■12,500
23331a42002	Priya Patel	IT	9.12	93%	■8,000
23331a42013	Meera Kulkarni	CSE	9.60	95%	Paid
23331a42003	Rohan Reddy	MECH	7.45	84%	■10,200
23331a42005	Vikram Gupta	CIVIL	6.82	78%	■15,000
23331a42006	Ananya Rao	EEE	9.40	96%	Paid

Staff dataset (data/staff.json)

20 members: 6 HODs, 10 Professors/Associates, 2 Deans, 2 Admin Officers. Each has staff_id, password, full_name, department, designation, email, phone, cabin_location, specialization, experience_years, office_hours.

Sample staff

ID	Name	Designation	Dept	Cabin
STF101	Dr. Rajesh Raman	Professor & HOD	CSE	CS-Block 301
STF117	Dr. Sudhir Joshi	Dean (Academics)	—	Admin Block
STF102	Dr. Sunita Kulkarni	Professor	IT	IT-Block 205

Data conversion (generate_json.py)

Reads CSV datasets and writes JSON files with full record structures. Allows editing CSV in Excel/Google Sheets and regenerating the runtime data without manually editing JSON.

7 Database Schema — supabase_schema.sql

23 186 bytes defining 9 tables with full DDL + seed INSERT statements. Row Level Security enabled on all tables (currently public access for development).

Table summary

Table	Purpose	Key fields / notes
departments	College departments	dept_code PK, name, building, hod_name
students	Student profiles	student_id, cgpa, attendance_pct, fee_balance
staff	Faculty profiles	staff_id, designation, cabin_location
subjects	Course catalog	subject_code, name, credits, faculty_id
student_attendance	Per-subject attendance	attended, total, percentage
fee_records	Payment tracking	tuition_fee, paid_amount, due_date
certificate_requests	Applications	certificate_type, status, applied_date
activity_logs	Timeline	title, description, icon
knowledge_base	AI Q&A; pairs	question, answer, keywords

Schema snippet — departments

```
CREATE TABLE IF NOT EXISTS public.departments ( dept_code VARCHAR(10) PRIMARY KEY, name VARCHAR(100) NOT NULL, building VARCHAR(100) NOT NULL, hod_name VARCHAR(100) NOT NULL );
```

REVIEW NOTE: All tables use RLS. Confirm policies restrict access to appropriate roles before exposing the endpoint publicly.

8 Dual Data Architecture

One design decision is deliberately highlighted: the app works online and offline with zero setup.

Without Supabase key

Reads students.json / staff.json locally. All dashboard, attendance, fee, and AI features work immediately. No network dependency.

With Supabase key

User enters anon key in Settings → `initSupabase()` activates → reads/writes to cloud tables. Local JSON remains as fallback. Settings screen shows pre-filled URL and empty key box with a **Test Connection** button.

The hybrid architecture is a strength for reviews: it shows awareness of offline-first design, cloud-sync readiness, and graceful degradation.

9 Authentication System

Simplified session model — not JWT-validated — for demonstration. Tokens are strings like `token_student_23331a42001_1695123456`.

Login behavior

Accepts student ID (23331a42001) OR email (aarav.sharma1@college.edu) — case-insensitive. Staff uses STF101 / rajesh.raman@college.edu. Password for all students: `student@123`. Password for all staff: `staff@123`. After login,

sessionStorage holds role + user; localStorage holds Supabase key (if entered). Login can fall back to local JS validation if server is offline.

Demo pills

Two 1-click buttons on login screen: ■ Aarav Sharma (CSE) and ■■■ Dr. Rajesh Raman (CSE HOD) for instant demo access without typing.

10 AI Chatbot Engine

Not an external AI model — a rule-based keyword matcher inside server.py that reads real student/staff data to answer context-specific questions.

How it works (from source)

Lowercase query → detect keywords (skip/miss/absent, fee/dues/balance, attendance/percentage, cgpa/marks, bonafide/certificate) → branch by role → find relevant records → compute projection (e.g., 28/36 → 28/37) → return formatted Markdown with emoji. Fallback is generic welcome.

Example responses

BMS today?	What is my fee balance?	Who is HOD of CSE?	Show students above 7 CGPA
BMS today?	Find DBMS attendance → project if skip 1 session	■■ 77.8% → 75.7% — attend next 2 to reach safe zone	
Fee balance?	Read fee_balance	■ Outstanding: ■12,500. Due Nov 15.	
HOD of CSE?	Search staff.json for CSE + HOD	■■■ Dr. Rajesh Raman — Cabin CS-Block 301	
Students above 7 CGPA	Filter by CGPA ≥ 7	■ 14 students found (Meera 9.60, Priya 9.12, ...)	

Staff-specific queries

Above / cgpa / top performer → list students by threshold. Send email / mail to → bulk email. Student / roster / shortage → department overview. Cabin / office hours → staff schedule.

11 API Endpoints

GET

Endpoint	Description
/api/config	App config + Supabase URL
/api/students[?dept=][?search=]	All students (filterable)
/api/students/23331a42001	Single student (full record)
/api/staff	All staff
/api/staff/STF101/courses	Staff courses + enrolled
/api/requests/all	All certificate requests
/api/stats	College statistics
/api/supabase-schema	Download SQL schema

POST

Endpoint	Body	Purpose
/api/auth/login	{role, identifier, password}	Authenticate
/api/chat	{message, user_id, role}	AI query

Endpoint	Body	Purpose
/api/fees/pay	{student_id, amount, method}	Pay fees
/api/requests	{student_id, certificate_type, purpose}	Apply cert
/api/requests/ID/approve	(body ignored)	Approve
/api/requests/ID/reject	(body ignored)	Reject
/api/attendance/mark	{student_id, subject_name, action}	Mark present/absent
/api/send-email	{staff_id, to_cgpa_above, ...}	Bulk email

12 Key Features & Demonstrations

Feature 1: Dashboard Overview

Welcome banner + 4 stat cards (attendance, CGPA, fees, requests) + subject attendance + recent activity + schedule + AI widget. Example: “Good morning, Aarav ■ — Attendance: 91% On Track — CGPA: 8.74 — Fees: ■12,500 Due Nov 15 — 1 Active Request.”

Feature 2: Fee Payment Simulator

Open Pay Fees → modal shows total + method → Confirm → server sets fee_balance=0, status=Paid → receipt REC-2509181430 → dashboard updates to “Fully Paid ■”.

Feature 3: AI Attendance Advice

Student with 77.8% DBMS attendance: skip → 75.7% (close to cutoff). Student with 92%: skip → 91.7% (safe). Both answers include projection + recommendation.

Feature 4: Staff Attendance Marker

Table with +Present / Absent buttons per student. Click updates backend → refreshes table. Integrated with department-level stats.

Feature 5: Certificate Requests

Student applies (type + purpose) → REQ-ID generated → status “In Review” → staff approves/rejects → instant update. Types include Bonafide, Provisional, Course Completion.

Feature 6: Staff AI Copilot

Dr. Radhika Menon (IT) asks “Show students above 7 CGPA” → 4 results with names, IDs, CGPAs, attendance %, emails. Follow-up “send email to students above 7” queues messages.

Feature 7: Digital Twin Health Score

Predictive score (0-100) from attendance consistency, grades, fee timing, lab milestones. Example: Aarav = 93/100 → “High probability of Distinction tier degree.”

13 How To Run & Testing

Quick start

```
# Navigate to project folder cd D:\test\smart-college-management # Start server (port 8080 by default) python3 server.py 8080 # Or: ./run.sh # Open browser # Visit: http://localhost:8080
```

Default accounts for review/demo

Role	ID / Email	Password
Student	23331a42001 / aarav.sharma1@college.edu	student@123
Student	23331a42006 / ananya.rao6@college.edu	student@123
Staff	STF101 / rajesh.raman@college.edu	staff@123
Staff	STF117 / dean.academics@college.edu	staff@123

Automated tests (test_app.py — 7 tests, ~0.5s)

Test	Verifies
01	CSV has 20 students; first Aarav Sharma; last Aditi Sharma
02	CSV has 20 staff across 7 departments
03	Student login by ID, by email, case-insensitive
04	Staff login by ID, by email
05	Invalid credentials rejected; unknown user rejected
06	SQL file has all required tables and seed data
07	AI responds correctly to attendance, fee, faculty, roster

All tests pass (expected: 7 dots, OK). Confirm test suite runs clean before final demonstration.

14 Appendix — Source Maps

Document provenance

This review PDF was generated from the actual project source at `D:\test\smart-college-management`. Source files read/verified: `PROJECT_DOCUMENT.md` (759 lines), `README.md` (128 lines), `server.py` (867 lines), `test_app.py` (131 lines), `run.sh` (13 lines), `data/*.csv + *.json + generate_json.py`, `supabase/supabase_schema.sql` (23 186 bytes), `static/index.html` (83 349 bytes), `static/js/app.js` (57 677 bytes).

Git & attribution (preserved verbatim)

Per project instructions, commit messages end with: `Co-Authored-By: Claude Code <noreply@anthropic.com>`. PR descriptions end with: `■ Generated with [Claude Code](https://claude.com/claude-code)`. These lines must be preserved exactly as specified.

Prepared for Smart College AI project review · Smart College AI Digital Twin OS · Version 2.5.0 · Supabase: <https://aqfqdjhdvypwccyp.supabase.co> · Default port 8080 · All content derived from actual source files.